

# Orgalyst: a conversational analysis system for organoid bright-field images

A domain toolkit that reproduces every number without any large language model, and an assistant that puts it in the hands of bench scientists.

**Category** End-to-End System

**Team** Orgalyst Lab: Xiaoyan Li (AI / computer vision / LLM applications, project lead); Liwen Xu and Xuzheng Fu (biology: organoid culture and morphological interpretation, responsible for requirements and review of results); Cuicui Jiang (AI: large language models, agent architecture and orchestration); Rumei Yang (biology: organoid analysis). Team composition: two members from AI / computer science and three from biology, as required for the cross-disciplinary bonus

**Code** <https://github.com/xiaoyanLi629/orgalyst> (MIT)

**Weights** <https://huggingface.co/XiaoyanLi/orgalyst-weights>

**Data** OrgLine (Zenodo 16355179, CC-BY-4.0); no private data was used

## 1 Abstract

Organoid and organ-on-a-chip experiments produce large numbers of bright-field micrographs every day, and the three questions asked most often are how many, how big, and has anything changed. Existing tools are either scripts that need tuning or software that segments but leaves statistics and record-keeping to the user, so turning a folder of images into a report that can go into a lab notebook still costs half a day. Orgalyst splits this chain into two layers. The lower layer is a deterministic toolkit, `orgalyst`: Cellpose segmentation with organ-specific weights, YOLO counting with calibrated confidence thresholds, morphometry, quality control, group statistics, growth curves, single-file HTML reports and a run manifest that records input hashes, weight hashes, parameters and software versions. The upper layer is a conversational assistant built on the Claude Agent SDK that calls these tools through MCP, so a bench scientist can go from images to report in one sentence while every number is still computed by the toolkit. We ran five groups of experiments on the public OrgLine dataset (eight sources, five organs). After per-organ fine-tuning, segmentation AP50 reaches 0.967, 0.812 and 0.815 on brain, intestine and colon (zero-shot cyto3: 0.005, 0.544, 0.405). A three-organ YOLO11m detector reaches mAP50 0.936 to 0.995, and calibrating the counting threshold on the validation set lowers the counting error on lung organoids from 0.235 to 0.168. On 240 brain organoid images the automatic area agrees with expert annotation at Spearman 0.963 with a median relative error of 1.9%. In an end-to-end benchmark of 12 natural-language tasks the assistant passed 12 of 12 at an average of 113 s per task. We also report honestly where cross-organ generalisation ends, where the system falls short on pancreatic cancer organoids, and why a Cellpose-SAM comparison brought no gain.

## 2 Problem definition and application scenario

Organoids are miniature tissues grown from stem cells in a three-dimensional environment; organ-on-a-chip devices place such tissues into microfluidic hardware that mimics physiological conditions. In both cases the cheapest, least invasive and most frequent way to observe them is the bright-field microscope: take a picture every day or every few hours and see how they are growing. The analysis these pictures need is very fixed. Count how many organoids are in an image, measure the size and shape of each, compare treatment groups, connect measurements of the same organoid over time into a growth curve, and finally write it all up as a report that can go into the lab record and be checked by someone else.

The bottleneck is not the algorithm. Tools such as Cellpose, OrganoID and OrgaSegment already segment organoids well, and the OrgLine dataset has even unified eight public sources into one format. What actually stops bench scientists are three things. First, the tools are for people who write code: an environment has to be installed, a model chosen, a diameter parameter tuned, and when organoids in one image differ tenfold in size the default parameters fail while the scientist does not know which knob to turn. Second, segmentation is only the first step; the measuring, statistics, plotting and report writing that follow are reinvented in every lab, inconsistently, so results cannot be compared. Third, nothing is recorded: three months later a reviewer asks which model version and which parameters produced this area, and most people cannot answer.

Orgalyst targets biologists who run organoid or organ-on-a-chip experiments and do not write code. Success is defined by three criteria: a scientist gets a complete analysis report using natural language alone; every number in that report can be recomputed from the command line in an environment without any language model, and agrees with the expert annotation shipped with a public dataset; and the system says when it is unsure rather than returning a wrong number that looks precise. The third criterion is why quality control and the run manifest are first-class parts of the toolkit rather than add-ons.

## 3 Data

All experiments use **OrgLine** (Zenodo 16355179, CC-BY-4.0), which unifies eight published organoid image datasets into an instance segmentation subset (InstanceSeg) and a detection subset (OrgDet). Segmentation covers four organs: brain (Schröter et al. 2024), intestine (OrgaSegment), pancreatic ductal adenocarcinoma, pdac (OrganoID and OrganoIDNet) and colon (OrgaExtractor). Detection covers intestine (Tellu, OrgaQuant, OrgaSegment), brain and lung (Deep-LUMEN). We used no private data from any company or laboratory and added no annotations.

`scripts/prepare_data.py` converts the raw data into 8-bit PNG images and uint16 instance masks (one integer label per organoid) plus YOLO text labels for detection; true-colour bright-field images keep RGB, images whose three channels are identical become grey, and 16-bit images are stretched to 8 bits between the 1st and 99th percentiles. The segmentation subset keeps OrgLine's own split; the detection subset is split 70 / 15 / 15 at random (seed 0) and the split files ship with the repository. A physical pixel size is documented only for the brain dataset (3.1646  $\mu\text{m}/\text{px}$  for Lab A images 1024 px wide, 2.9940  $\mu\text{m}/\text{px}$  for Lab B images 1388 px wide). The other source papers give the instrument and objective but no calibration; we do not guess, so morphometry for those sources is reported in pixels and the report says so.

| Segmentation subset (images / instances) | train       | val         | test       |
|------------------------------------------|-------------|-------------|------------|
| brain                                    | 1431 / 1431 | 186 / 186   | 240 / 240  |
| intestine                                | 184 / 11977 | 35 / 2551   | 12 / 973   |
| pdac                                     | 765 / 52281 | 193 / 12296 | 20 / 10268 |
| colon                                    | 15 / 1487   | 5 / 0       | 10 / 750   |

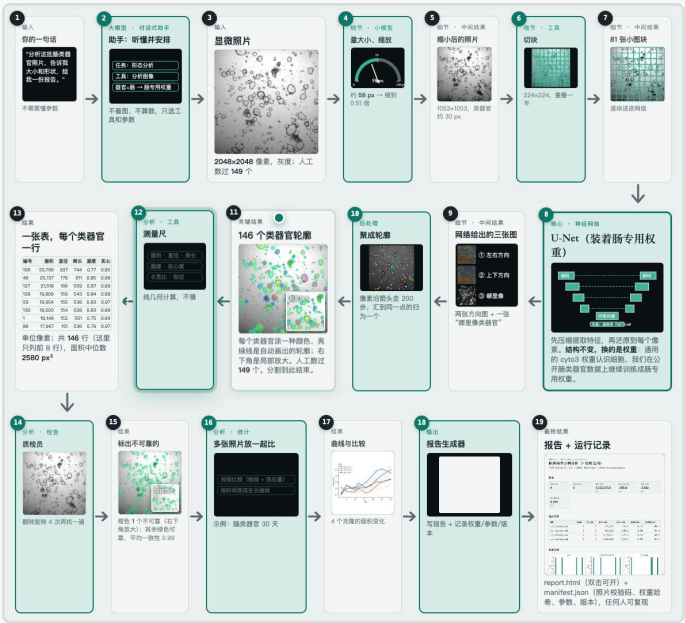
| Detection subset (images / boxes) | train        | val         | test        |
|-----------------------------------|--------------|-------------|-------------|
| intestine                         | 1969 / 36914 | 421 / 7631  | 423 / 8443  |
| brain                             | 1299 / 1299  | 278 / 278   | 280 / 280   |
| lung                              | 2804 / 51217 | 600 / 10903 | 602 / 10702 |

Each brain image contains exactly one organoid (the 240 test images come from 4 clones  $\times$  3 individuals  $\times$  10 time points  $\times$  2 laboratories), which makes it natural material for growth curves and for the comparison with expert-annotated area. The pdac test images contain more than 500 small instances each while training images average 68; this in-domain shift is the main source of failure discussed in sections 7 and 8. Colon has only 15 training images and no masks in its validation set.

## 4 Methods and system

### 4.1 Architecture

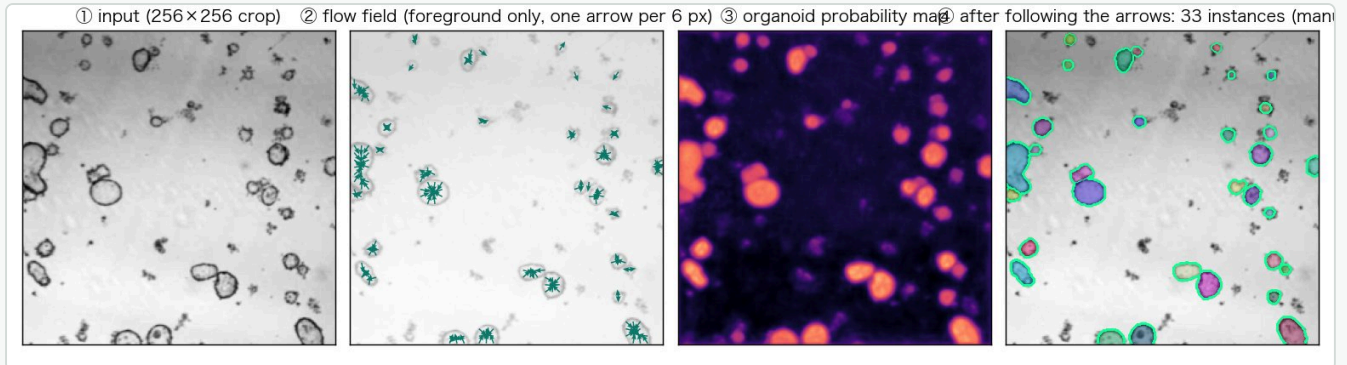
The system has two layers. The **toolkit layer**, `orgalyst`, is an ordinary Python package with the command-line entry point `python -m orgalyst analyze | count | track | report` and needs no language model. Every run leaves a directory containing `manifest.json` (md5 of each input file, model identifier and weight hash, all parameters, numpy / torch / cellpose / ultralytics versions, GPU model, step-by-step log), `tables/` (per-instance features, summary, QC, comparison and growth tables), `masks/`, `overlays/` and a single-file `report.html`. The **assistant layer** is a slimmed copy of BioAgent, the Claude Agent SDK assistant we built earlier for our laboratory (permission gatekeeper, terminal and web interfaces). The toolkit is exposed to it through one MCP server (`orgalyst/mcp_server.py`, stdio transport) with six tools: `list_models`, `analyze_images`, `count_organoids`, `compare_groups`, `growth_curves`, `run_summary` (plus `list_runs`). The assistant's system prompt states which tool to use when, that answers must distinguish pixels from micrometres, and that border-touching instances are excluded by default. The assistant does not look at pixels or compute numbers; it only interprets the request, chooses parameters and explains the results.



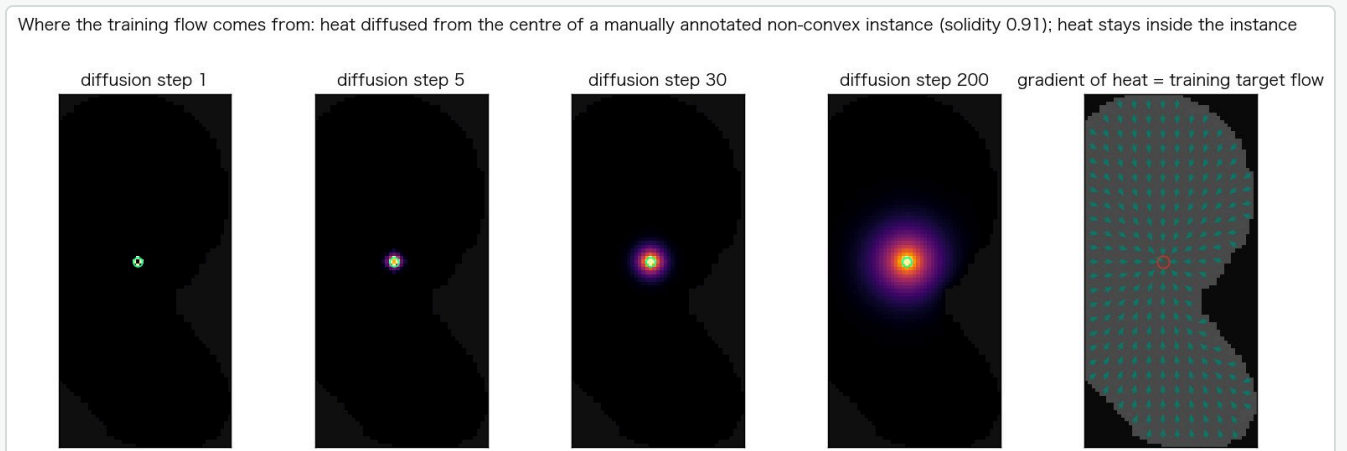
**Figure 1 · The complete journey of one intestinal organoid image.** White cards are data, green boxes are models or tools, large cards are the key steps. From the user's sentence to the assistant choosing tools, then size estimation, rescaling, tiling, the U-Net loaded with intestine-specific weights, the three network outputs, tracking into contours, 146 organoid outlines, the measurement table, QC, statistics, the report generator, and finally the report plus the run record. Labels are in Chinese; the interactive version is `docs/orgalyst_scene.html`.

## 4.2 Segmentation: the Cellpose architecture and organ-specific weights

Segmentation uses Cellpose (Stringer et al., Nature Methods 2021). Its core is not a particular network but a reformulation of instance segmentation as regression of a flow field. At training time, heat is diffused from the centre of every annotated instance, giving each pixel the direction towards the centre of the instance it belongs to. A U-Net with residual blocks and skip connections learns to predict these two direction maps and one map of whether a pixel is an object at all. At inference every foreground pixel is moved along the direction map for about 200 steps, and pixels that converge to the same point form one instance. This design handles touching, non-convex and very differently sized objects naturally, and it makes the backbone replaceable: Cellpose-SAM is the same group's 2025 version with a ViT backbone.

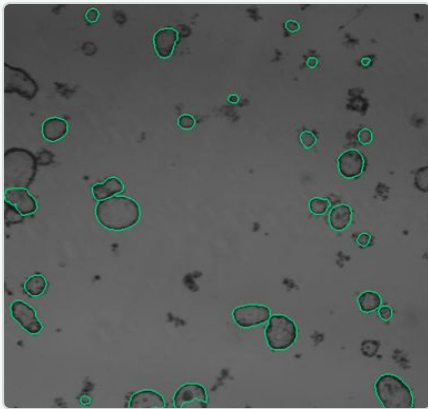


**Figure 2a · What the network actually outputs, and the tracking step.** A 256×256 crop of an intestinal organoid test image: the network outputs two direction maps (drawn here as arrows) and one probability map; pixels with probability above 0 follow the arrows for 200 steps and those that land on the same point form one instance, giving 33 instances against 35 annotated. Two touching organoids are separated because their arrows point to different centres.

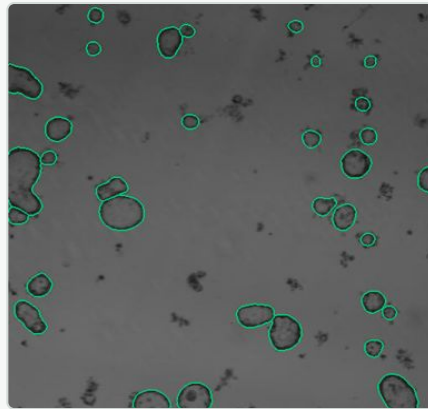


**Figure 2b · Where the training target comes from.** Nobody draws arrows: for every annotated instance, heat is diffused repeatedly from its centre (heat only travels inside the instance; each step averages the four neighbours). After a few dozen steps the heat field follows the instance shape, and its gradient is the direction each pixel should point to. This depends only on the annotation mask, so any dataset with instance masks can be used for training.

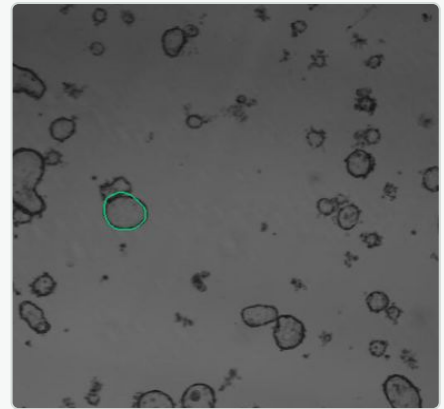
Cellpose has an easily overlooked precondition: the network is trained at a scale where objects are about 30 pixels wide, so before inference the typical diameter of objects in the image must be estimated and the image rescaled accordingly. Getting the diameter wrong has direct consequences. Too small, and the image is enlarged so one organoid is split into several; too large, and the image shrinks so all organoids blur into one. Cellpose ships a size model that regresses the diameter from the network's style vector, but it was trained on cell images and predicts about 30 pixels for brain organoids 400 pixels wide, which is why zero-shot brain AP50 is only 0.005.



Diameter set to 12 px (too small): 27 found, large organoids fragmented



Automatic estimate 29 px: 33 found, 35 annotated

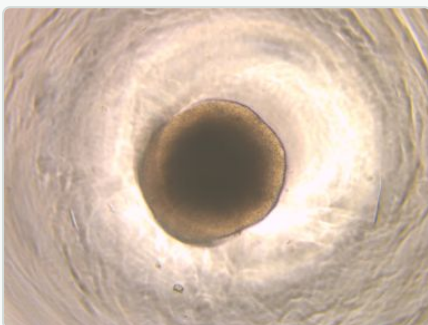


Diameter set to 150 px (too large): only 1 found

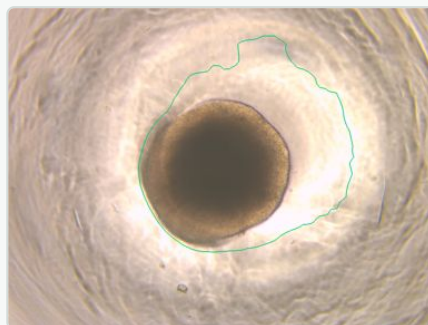
Figure 2c · Same intestinal organoid image, same model, only the diameter estimate changed.

Our approach is to **load organ-specific weights and pick a diameter strategy per organ**. The weights come from continuing to train the generalist cyto3 on each OrgLine organ for 200 epochs (section 6); during training Cellpose stores the median instance diameter of the training set in the weights ( `diam_labels` ). Four diameter strategies are registered per organ in

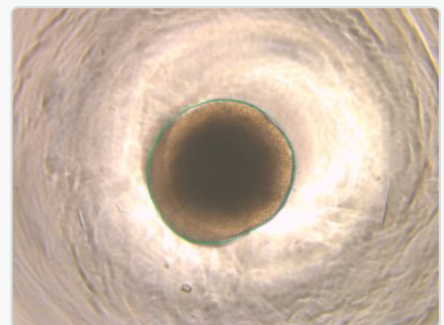
`orgalyst/config.py` : `model` uses the training diameter stored in the weights (colon); `sizemodel` estimates the diameter per image with the cyto3 size model (intestine and pdac, whose images differ in magnification); `refine` runs inference twice, first at the training diameter and then at the median equivalent diameter of the objects found in the first pass, clamped to 0.25 to 4 times (brain, whose organoids grow almost fivefold in area between day 2 and day 30, so no single diameter works at both ends); `per_image_gt` uses the ground-truth diameter and serves only as an experimental upper bound. Unknown organs fall back to zero-shot cyto3 with the size model, and the report marks the result as "generalist model, lower accuracy".



Generalist cyto3 + automatic diameter (estimated 30 px): 0 found



Generalist cyto3 + diameter set to 404 px: 1 found



Brain-specific weights (diameter stored in the model): 1 found

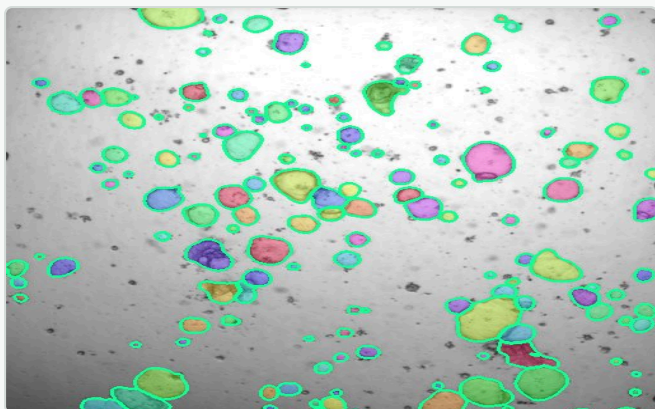
Figure 3 · On brain organoids the generalist model fails because of scale, not appearance; the organ-specific weights remember the scale.

### 4.3 Counting by detection

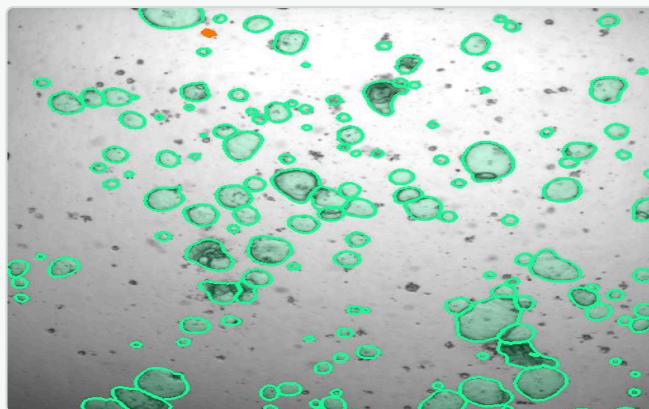
When the user only asks how many, segmentation is wasteful: a 2048-pixel intestinal image takes seconds to segment and well under a second to detect. We trained YOLO11m on the OrgLine detection subset (input 1024, 100 epochs) and compared three single-organ models with one three-organ joint model; the joint model matched or beat every single-organ model, so the system keeps only that one. A parameter that is easy to overlook in counting is the confidence threshold: the ultralytics default of 0.25 is designed for computing mAP and deliberately keeps many low-confidence boxes, which over-counts by about twenty percent when used directly. We swept the threshold from 0.05 to 0.90 on each organ's validation set and kept the value with the lowest mean relative counting error (intestine 0.50, brain 0.45, lung 0.40; 0.45 for unknown organs, the pooled optimum), wrote it into the configuration, and used the test set only to report the final error.

### 4.4 Morphometry, quality control, statistics and growth curves

**Measurement** uses scikit-image regionprops: area, equivalent diameter, perimeter, circularity ( $4\pi \cdot \text{area} / \text{perimeter}^2$ ), solidity (area / convex-hull area), aspect ratio and whether the instance touches the image border; when a pixel size is given the micrometre columns are added. Border-touching instances are incomplete, so summary statistics exclude them by default and the report states how many. **Quality control** works at two levels. Image level: sharpness (variance of the Laplacian), uneven illumination (coefficient of variation of local means) and the fraction of saturated pixels. Instance level: test-time augmentation with flips and rotations, comparing the segmentation of the original image with those of 4 dihedral transforms; an instance recovered in fewer than half of the transforms is flagged "low confidence" and drawn in orange in the report. The user thus receives not just a number but also which instances are doubtful. **Statistics** compare groups: Mann-Whitney U for two groups, Kruskal-Wallis for more, Cliff's delta as effect size, Holm correction across metrics, with box plots and a comparison table. **Growth curves** connect the per-image median area over individuals, time points and groups from metadata, giving each individual's fold change relative to its first time point and the mean curve per group.



**Figure 4a · Segmentation result.** A 2048×2048 intestinal organoid test image with 146 instances, each filled in its own colour with a bright green outline; 149 were annotated manually.



**Figure 4b · Quality control.** The same image segmented again after 4 flips and rotations; orange marks the instances whose agreement is below 0.5 (one), the rest are green; mean agreement 0.99.

## 4.5 Agent orchestration and record-keeping

On receiving a request the assistant first breaks the work into three to eight steps with its task-list tool and tells the user the plan, then calls the tools step by step. The judgements it actually has to make are few: which organ the images are (if unstated, use generic and say so), whether a pixel size is known (if not, report pixels and never invent one), whether the user wants counting or measurement, whether QC is wanted, and where group and time-point information lives. Every tool call writes into the output directory of the current conversation, and answers cite the run directory and the report path. The permission gatekeeper

( `agent/bioagent/permissions.py` ) keeps raw-data directories read-only and forbids access to other accounts' directories and to secrets; the user chooses a confirmation mode per conversation (confirm everything, confirm only dangerous commands, fully automatic, or read-only). The assistant also keeps a per-account long-term memory (which project the user is working on, usual data paths, pixel calibrations and preferences, summarised automatically at the end of a session), and the web edition supports multiple accounts with per-account spending limits and an administrator panel; memory was disabled during the E5 benchmark so that tasks stayed independent.

**Open-model path.** The Claude Agent SDK can be pointed through `ANTHROPIC_BASE_URL` at any gateway that speaks the Anthropic Messages API; `agent/README.md` documents a LiteLLM-in-front-of-vLLM configuration, and the E5 benchmark script can be run once per backend. For lack of time this report gives results for the Claude backend only, which section 8 lists as a limitation.

## 4.6 Usage flow and outputs

A typical session goes like this. The scientist types, in the web or terminal interface, "analyse the 4 intestinal organoid images under /data/exp3, do a morphological analysis and generate a report". The assistant lists its plan (confirm organ and pixel size, call the analysis tool, read the summary, write the answer), then calls `analyze_images(images="/data/exp3", organ="intestine", name="exp3")`. The toolkit segments, measures and builds the report within tens of seconds and returns the run directory and summary; the assistant turns the total, per-image counts, and the medians and interquartile ranges of area and circularity into a table and gives the report path. The end of the assistant's answer to this task in the E5 benchmark reads as follows (excerpt, translated). It states the units, points out that it did not invent a pixel size, suggests checking the overlays first, and says plainly that "201 is a model output with no accuracy guarantee", which is exactly the judgement we want the assistant to show:

"Note: all sizes are in pixels, not micrometres. You did not provide pixel\_size\_um and I have not invented a calibration. ... I suggest looking through the 4 overlay images in overlays/ first. This run used the intestine-specific model, but without manual annotation as a gold standard, 201 is still just a model output with no accuracy guarantee."



**Figure 4c · The single-file report.html produced by the toolkit (the intestinal organoid batch of E5 task 2, 4 images, 201 instances; interface language Chinese).** Overview tiles (count, medians of area / diameter / circularity), the per-image table, distribution histograms, overlay thumbnails, a templated methods paragraph (model, diameter strategy, exclusion rules) and the provenance table (input hashes, weight hash, versions). The report needs no network, opens with a double click, and can be attached directly to a lab record.

## 5 As an analysis assistant: what can be asked, and how it works

Seen together, the toolkit and the assistant present the user with one assistant that knows how to analyse organoid images: the scientist describes the task in plain language, the assistant follows a standard procedure to call the tools, checks the results, reports in a fixed format, and leaves the report and data in the output directory of the conversation. It is not a chatbot with a few functions attached; it is an analysis protocol built into an assistant. Below we describe what can be asked, what procedure is followed, and what the interface looks like.

### 5.1 What can be asked

| How the user asks                                                      | What the assistant does                                                               | Tools used                                     | What the user gets                                                                            |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------|
| "How many organoids are in these images?"                              | Counts per image with the detector, threshold taken from the organ's calibrated value | <code>count_organoids</code>                   | Per-image counts table, image boxes drawn, note on the range                                  |
| "Analyse these intestinal organoid images, tell me size and shape"     | Segments with the organ's weights, measures every instance, builds the report         | <code>analyze_images</code>                    | Total, per-image table, median and IQR of diameter / circularity, report.html, features.csv   |
| "Pixel size is 3.16 $\mu\text{m}$ , give me areas in $\mu\text{m}^2$ " | Passes the calibration to the tool; all lengths and areas reported in micrometres     | <code>analyze_images(pixel_size_um=...)</code> | Results in micrometres without a calibration<br>"pixels only never estimate one"              |
| "Which segmentations are unreliable? Is image quality OK?"             | Turns on QC: image-level metrics plus flip/rotate agreement                           | <code>analyze_images(qc=True)</code>           | Sharpness, illumination, saturation, low-confidence instances, not in orange with their count |
| "Is there a difference in area between control and treated?"           | Non-parametric tests and effect sizes from a group file                               | <code>compare_groups</code>                    | Medians per group, p value, Cliff's delta plot, compare.html, plain-language conclusion       |
| "How much did it grow from day 2 to day 30?"                           | Connects individuals over time points into growth curves                              | <code>growth_curves</code>                     | First and last values and change per individual, regression curve per group, growth.html      |
| "This batch has both intestine and brain, analyse them separately"     | Splits the batch and uses the matching weights for each                               | two <code>analyze_images</code> calls          | Two run directories, their own counts                                                         |

|                                                                  |                                                                                     |                                                   |                                                                                               |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                                                  |                                                                                     |                                                   | and morpho<br>summaries                                                                       |
| "How is circularity computed? Why exclude border-touching ones?" | Explains definitions and rules without recomputing anything                         | (none)                                            | A two- or th<br>sentence<br>explanation                                                       |
| "Which model and parameters were used? How do I reproduce this?" | Reads the run manifest                                                              | <code>run_summary</code> / <code>list_runs</code> | Path of<br>manifest.js<br>what it reco<br>input hashe<br>weight has<br>parameters<br>versions |
| "Analyse the images in /data/x" (empty folder / organ unstated)  | Says no images were found; or uses the generalist model and declares lower accuracy | —                                                 | One clear<br>statement i<br>of a plausib<br>looking nur                                       |

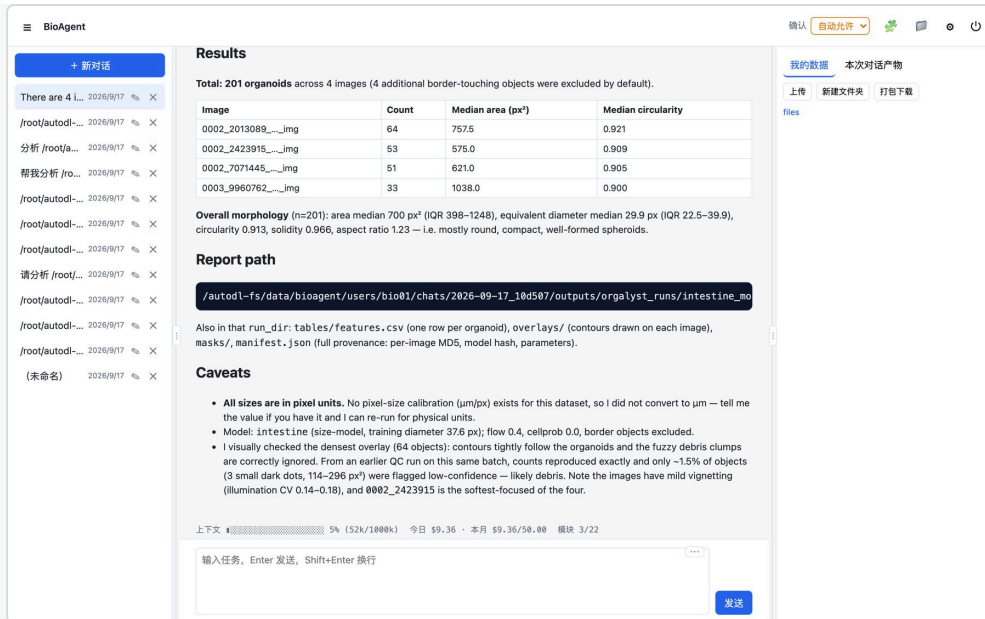
Table 5-1 · Question types the assistant covers. The first ten rows are exactly the range covered by the twelve E5 tasks (section 7.5).

### 5.2 What procedure is followed

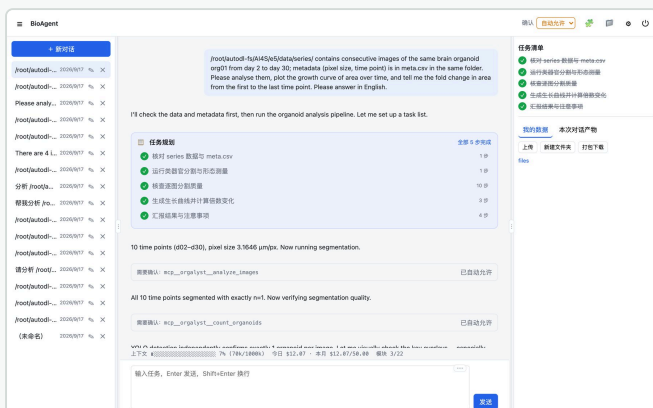
Tools alone are not enough. When the assistant should ask what, what to run first, how to check and how to report is written down as a reusable procedure, `agent/skillpack/skills/organoid-analysis/SKILL.md`, loaded automatically as a local skill of the Claude Agent SDK. It activates whenever the user mentions organoid image analysis and can also be invoked explicitly with `/organoid-analysis`. It prescribes four steps. First, confirm three prerequisites (organ type, pixel size, and whether the user wants counts, morphology or a comparison; ask for whatever is missing, but do not block a simple task). Second, choose tools from those prerequisites (counting goes to detection, everything else to segmentation; enable QC for small batches or when the user cares about reliability; attach statistics when group or time-point information exists). Third, check the returned values before answering (a per-image count of 0 or an outlier triggers a look at the overlay; the number of border-excluded and low-confidence instances must appear in the answer). Fourth, report in a fixed six-part format (one-sentence conclusion; per-image table with medians and interquartile ranges; units and exclusions; statistical conclusion; output paths; at most two next-step suggestions). The skill also lists prohibitions: no micrometre figures without a pixel size, no recomputing in Python what a tool already reported, no re-running the same batch, no self-written segmentation or measurement code; the last rule is also enforced by the permission gatekeeper (section 8). The assistant's behaviour therefore no longer depends on how well each prompt is written, and the same procedure carries over to a different model backend.

The effect shows in two ways. Answers have a fixed skeleton: prerequisites first (organ, units), then the conclusion and table, then paths and suggestions, as in the answer quoted in section 7.5. And the assistant says no when it should: no micrometre numbers without a pixel size, generalist-model results labelled as lower accuracy, outlying per-image counts checked against the overlay instead of copied. These behaviours live in the skill and the system prompt, not in the user's ability to ask well.

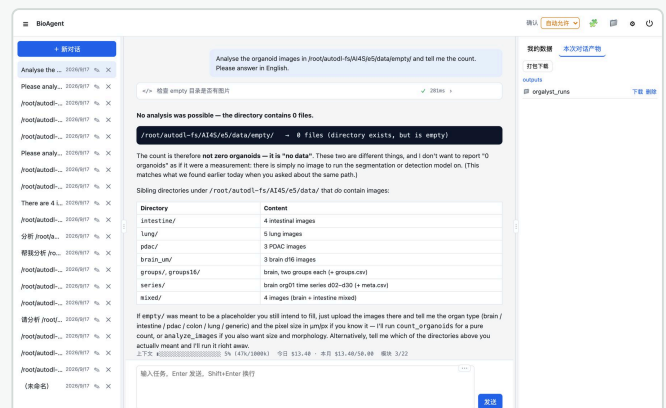
### 5.3 Interface



**Figure 5-1 · A real analysis in the web interface (account bio01, 4 intestinal organoid images; request and answer in English, interface chrome in Chinese).** The centre shows the assistant's answer: prerequisites, per-image count table, overall morphology, output paths and suggestions. Top right is the confirmation mode of this conversation ("auto-allow" here); the right column shows the task list the assistant maintains and the files produced in this conversation; the status bar at the bottom shows context usage, today's and this month's spending, and the loaded tool modules. The same backend also has a terminal edition for scripted use and evaluation, which is how E5 was run.



**Figure 5-2 · The start of a growth-curve analysis.** The assistant states its plan, creates a task list (mirrored on the right with progress), and every tool call leaves an "auto-allowed" record in the conversation; the left column lists the account's previous conversations.



**Figure 5-3 · Files produced in this conversation.** The right panel lists the run directory, tables and figures generated in the conversation, with preview, download and zip; the "my data" tab is the account's own upload area.



**Figure 5-4 · Tool module panel.** Besides Orgalytst, the assistant can load 22 groups of Biomni bioinformatics tools on demand (database queries, literature, molecular biology and more); only three are loaded by default to keep each turn cheap.



**Figure 5-5 · Settings page.** Model choice, the default confirmation mode for new conversations, the per-account long-term memory (summarised automatically at the end of a session, editable or clearable by the user) and password change.

## 6 Implementation details

Hardware: one AutoDL cloud instance with an NVIDIA RTX 5090 D (32 GB) and a container memory limit of 62 GB. Software: Python 3.10, torch 2.14.0+cu130, cellpose 3.1.1.3, ultralytics 8.4.152, numpy 2.0.2, scikit-image 0.25. Segmentation fine-tuning: initialised from cyto3, 200 epochs per organ, learning rate 0.1 (Cellpose's default SGD), batch size 8; wall time 74 min for brain, 31 for pdac, 22 for intestine, 1.4 for colon. The cross-organ experiments cap each organ at 500 training images (the full four-organ set of 2395 images exceeds the container memory and is killed silently, one of the pitfalls we hit). Detection: initialised from yolo11m.pt, input 1024, batch 8, 100 epochs; 32 to 65 min per single-organ model and 126 min for the joint model. The Cellpose-SAM comparison ran in a separate cellpose 4 environment with batch 2, learning rate 1e-5, 100 epochs: 96 min for pdac and 25 for intestine. Measured per-stage timings are in Table 6-1.

| Stage (single image, median)                           | 512px (51 instances) | 1024px (67 instances) | 2048px (55) |
|--------------------------------------------------------|----------------------|-----------------------|-------------|
| Segmentation (organ weights + per-image size estimate) | 0.29 s               | 0.71 s                | 2.29 s      |
| QC: flip/rotate agreement (4 passes)                   | 1.06 s               | 2.94 s                | 9.07 s      |
| QC: image-level metrics                                | 0.05 s               | 0.40 s                | 3.14 s      |
| Morphometry + summary                                  | 0.03 s               | 0.06 s                | 0.11 s      |
| Overlay                                                | 0.01 s               | 0.02 s                | 0.09 s      |
| Detection count (YOLO11m)                              | 0.01 s               | 0.01 s                | 0.01 s      |
| Peak GPU memory                                        | 0.7 GB               | 0.7 GB                | 0.7 GB      |

Table 6-1 · Per-stage timing (scripts/bench\_timing.py, results/timing.json; RTX 5090 D, intestine weights + sizemodel). On CPU only, segmenting the 512px image takes 14 s, still usable for small batches. Report

generation (histograms and thumbnails) adds about 2 to 5 s.

The MCP interface is implemented with FastMCP; tool signatures are as follows (all parameters have defaults, all return JSON):

```
analyze_images(images, organ="generic", pixel_size_um=None, name, diam_mode=None, limit=0, make_report=True, qc=False, meta_csv=None) ; count_organoids(images, organ="generic", name, conf=None, limit=0) ; compare_groups(run_dir, groups, metrics=None, exclude_border=True) ; growth_curves(run_dir, meta_csv=None, pattern=None, metric=None) ; run_summary(run_dir) ; list_models() ; list_runs(limit=20) . At start-up the MCP server redirects stdout to stderr and keeps a private handle for the protocol stream, so that print statements inside the underlying libraries cannot corrupt the protocol, a practice we adopted when integrating the Biomni tool library.
```

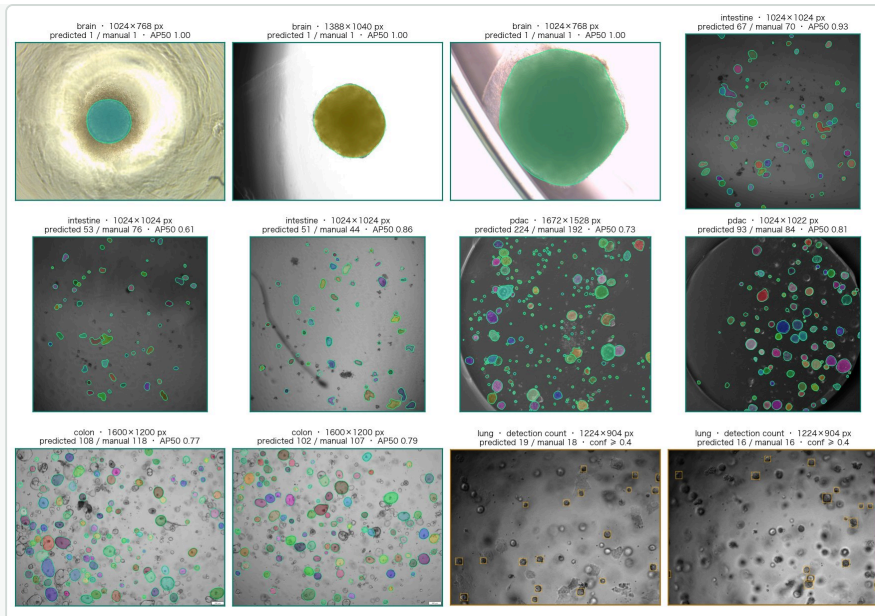
## 7 Experiments and results

### 7.1 E1 segmentation: zero-shot, per-organ fine-tuning, diameter strategy and cross-organ generalisation

The metric is instance-level AP50 (a predicted instance matches a ground-truth instance at IoU  $\geq 0.5$ , computed with `cellpose.metrics.average_precision`); AP75 and AP90 are also reported to reflect boundary quality. Table 1 gives the main results.

| Organ (test images) | cyto3 zero-shot | Per-organ fine-tuned (model diameter) | Fine-tuned |
|---------------------|-----------------|---------------------------------------|------------|
| brain (240)         | 0.005           | 0.925                                 | 0.967      |
| intestine (12)      | 0.544           | 0.773                                 | 0.812      |
| colon (10)          | 0.405           | 0.815                                 | 0.814      |
| pdac (20)           | 0.419           | 0.564                                 | 0.591      |

Table 1 · E1 segmentation AP50. Zero-shot cyto3 uses its own size model; the "upper bound" gives each image its ground-truth median diameter, which is not available in practice and only separates "scale not estimated" from "model does not recognise".



**Figure 7-1 · The default pipeline on test images (12 images, unselected: the first few of each organ).** The three brain images are different clones, days and laboratories; the organ weights plus two-pass inference outline the whole organoid accurately. Intestine, pdac and colon are multi-instance images with predicted instances filled in colour; titles give predicted count, manual count and the image's AP50. The last two are lung organoid detection counts with orange boxes. Typical error sources are visible: small fragments missed in the second intestine image (AP50 0.61), more predictions than annotations in the first pdac image (224 vs 192), and border-touching and adjoining instances in colon.

Three conclusions. First, zero-shot cyto3 recognises organoids (colon recall 0.985) but precision is only 0.4 to 0.6, over-segmentation is severe and AP75 is 0.2 to 0.4; the main failures are scale estimation and boundary quality, and fine-tuning fixes both. Second, the diameter strategies pay off: two-pass inference takes brain from 0.925 to 0.967, almost the ground-truth-diameter bound, and per-image size estimation takes intestine from 0.773 to 0.812. Third, pdac remains weak: test images hold more than 500 small instances each, recall is about 0.5, AP90 is near zero, and even the ground-truth diameter only reaches 0.605, so the problem is the training distribution (68 instances per training image on average, test images from a different instrument) rather than scale.

**Cross-organ generalisation.** We trained one four-organ joint model and four leave-one-organ-out models. Given the right scale, the joint model matches the specialised ones (with GT diameter: intestine 0.810, pdac 0.606, brain 0.971, colon 0.789; with the usable sizemodel strategy: intestine 0.797, pdac 0.610, colon 0.789, brain refine 0.896), but the training diameter it stores, 183 pixels, is wrong for every single organ, so the model strategy almost completely fails (0.04 to 0.42). Leave-one-out models reach only 0.15 to 0.47 on an organ they have never seen even with the right scale (colon best at 0.465 because it resembles intestine; refine 0.580). Cross-organ generalisation is therefore limited, and organ-specific weights are a necessity rather than a design preference; the joint model serves only as a fallback for unknown organs, where it beats zero-shot cyto3.

**Cellpose-SAM comparison.** Cellpose-SAM (cellpose 4, ViT-L backbone, advertised as needing no diameter) reaches zero-shot AP50 of about 0.60 on intestine, 0.43 to 0.48 on pdac, about 0.03 on brain and about 0.37 on colon, the same level as zero-shot cyto3, and fails on brain in the same way. After 100 epochs of per-organ fine-tuning it reaches 0.557 on pdac (0.591 with GT diameter) and 0.780 on intestine (0.795), matching but not exceeding fine-tuned cyto3, with weights of 1.2 GB (26 MB for fine-tuned cyto3) and twice the inference time. We therefore keep fine-tuned cyto3 as the main model; the comparison results are preserved in full under results/.

## 7.2 E2 counting by detection

| Organ (test images) | Joint model mAP50 | mAP50-95 | Single-organ mAP50 | Calibrated tl |
|---------------------|-------------------|----------|--------------------|---------------|
| intestine           | 0.948             | 0.739    | 0.946              | 0.5           |
| brain               | 0.995             | 0.941    | 0.995              | 0.45          |
| lung                | 0.936             | 0.754    | 0.931              | 0.4           |

Table 2 · E2 detection. Count MAPE is the mean over images of  $|\text{predicted} - \text{true}| / \text{true}$ ; total bias is predicted total / true total. Intestine 423 images, brain 280, lung 602.

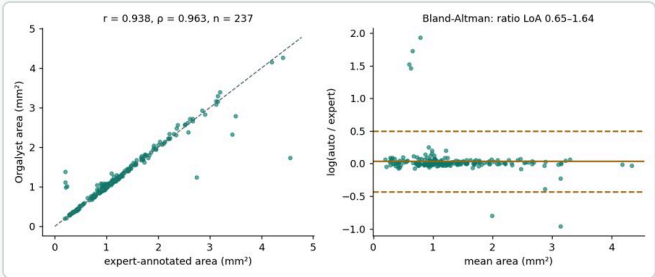
The joint model matches or slightly beats the single-organ models, so the system keeps one detector. Threshold calibration lowers the counting error on intestine and lung by twenty to thirty percent and corrects the systematic over-count of 10 to 12% at the default threshold to a total bias near 1; brain has one organoid per image and near-zero error at any threshold. The remaining error comes mainly from adjoining clusters and half-organoids at the image edge; a median relative error of 0.11 to 0.13 means most images are off by one or two.

### 7.3 E3 agreement with expert annotation

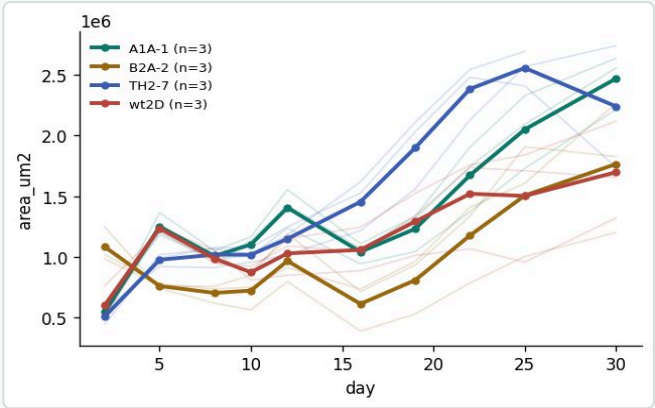
The original brain organoid dataset (Schröter et al. 2024) provides, besides masks, a manual area in pixels<sup>2</sup> per image; we verified that it equals the area of the OrgLine ground-truth mask image by image, so it can serve as an expert reference. We ran the system's default pipeline (brain weights + refine) on the 240 test images and compared automatic with expert area.

| Diameter strategy               | Detected / missed | Pearson r | Spearman ρ | Median ratio | Median |
|---------------------------------|-------------------|-----------|------------|--------------|--------|
| model (fixed training diameter) | 230 / 10          | 0.921     | 0.959      | 1.009        | 2.2%   |
| refine (system default)         | 237 / 3           | 0.938     | 0.963      | 1.010        | 1.9%   |

Table 3 · E3 automatic vs expert-annotated area, brain test set, 240 images.



**Figure 5 · Scatter and Bland-Altman plots of automatic vs expert area (refine).** The residual failures are the smallest day-2 organoids (4 images over-estimated 4 to 7 times because the first pass took the medium background for the object) and one of the largest day-30 organoids under-estimated by about half.



**Figure 6 · Area growth curves of the four clones.** Generated automatically from the same 240 analysis results after parsing individual, clone and time point from the file names; thin lines are individuals, thick lines group means. Axis labels in English.

### 7.4 E4 what quality control contributes

The value of the QC module is that it marks errors rather than removing them. Does it mark the right ones? We tested this on the three organ test sets that have ground-truth masks: for every predicted instance we computed its maximum IoU with any ground-truth instance, called it a wrong instance when IoU was below 0.5 (spurious, fragmented or badly displaced), and looked at how the "low confidence" flag (flip/rotate agreement below 0.5) relates to wrong instances.

| Organ     | Predicted instances | Flagged low-confidence | Wrong instances (IoU<0.5) |
|-----------|---------------------|------------------------|---------------------------|
| intestine | 923                 | 10                     | 87                        |
| colon     | 709                 | 17                     | 56                        |
| pdac      | 5857                | 148                    | 876                       |

Table 7-3 · E4 QC flag vs actual segmentation quality (scripts/e4\_qc\_eval.py, results/e4\_qc\_eval.json). Precision is the share of flagged instances that are wrong; recall is the share of wrong instances that were flagged;  $\rho$  is the rank correlation between agreement and IoU.

On all three organs the instances flagged as low-confidence are indeed much worse: the share of wrong instances among them is 3.5 to 6.8 times that of the remaining instances, and their mean IoU is far lower, so flip/rotate agreement is a genuine reliability signal. Recall, however, is low (0.07 to 0.12): most wrong instances are segmented wrongly in a stable way under flips and rotations, and the many small pdac organoids that are missed altogether never appear as instances at all, so the flag cannot catch what is missing, only what is unstable. The report therefore presents QC as a hint rather than a filter: it directs the user's review effort to the most suspicious instances instead of deciding for the user what to delete.

On the intestinal example image, 120 instances have a mean flip/rotate agreement of 0.98 and only one is flagged, the truncated cluster in the image corner; in the E5 QC task, 3 of 205 instances across 4 intestinal images were flagged.

## 7.5 E5 end-to-end agent benchmark

We designed 12 natural-language tasks covering single-image analysis, batch analysis with report, analysis with physical pixel size, counting only, group comparison, growth curve, quality control, a follow-up about metric definitions, a follow-up about reproducibility, an unspecified organ, an empty folder and a mixed-organ batch. Each task uses a few of the 36 test images drawn from OrgLine, and success is judged automatically by a script: numbers in the answer must lie within 5 to 10% of the reference values computed by the command-line path, the agreed output files must exist, and key wording must be present (an explanation of circularity must mention the perimeter, an empty folder must be reported as containing no images); for the unspecified-organ task either asking the user or using the generalist model with a caveat is accepted. The runner drives the Agent SDK directly without a terminal, one fresh conversation per task (the two follow-ups run inside their parent task's conversation), with the confirmation mode set to fully automatic.

| Task | Content                                 | Verdict | Time (s) | Tool calls | of which Orgalyst | Cost (\$) |
|------|-----------------------------------------|---------|----------|------------|-------------------|-----------|
| t01  | single image:<br>count and<br>size      | pass    | 49       | 2          | 1                 | 0.13      |
| t02  | batch<br>analysis with<br>report        | pass    | 91       | 13         | 3                 | 0.32      |
| t08  | follow-up:<br>what the<br>metrics mean  | pass    | 21       | 1          | 0                 | 0.43      |
| t03  | analysis with<br>physical pixel<br>size | pass    | 63       | 2          | 1                 | 0.14      |
| t10  | follow-up:<br>reproducibility           | pass    | 29       | 2          | 0                 | 0.27      |
| t04  | count only<br>(detection)               | pass    | 36       | 2          | 1                 | 0.09      |
| t05  | group<br>comparison                     | pass    | 258      | 27         | 6                 | 0.91      |
| t06  | growth curve                            | pass    | 259      | 28         | 5                 | 0.88      |
| t07  | quality control                         | pass    | 152      | 20         | 2                 | 0.58      |
| t09  | organ not<br>specified                  | pass    | 121      | 8          | 1                 | 0.39      |
| t11  | empty folder                            | pass    | 30       | 1          | 0                 | 0.09      |
| t12  | mixed-organ<br>batch                    | pass    | 82       | 13         | 3                 | 0.29      |

Table 4 · E5 results, Claude backend (claude-opus-5). Success rate 12/12, 9.9 tool calls per task on average (of which 1.9 are Orgalyst tools; the rest are the task list, file reads and directory checks), 99 s per task, 4.51 US dollars in total.

Table 4 is the full re-run after both the skill of section 5 and the code guard of section 8 were in place ( `results/e5_agent_tasks_claude_v2.json` ). The first run, before either, also passed 12/12 but at 113 s and 10.5 tool calls per task and 5.63 USD in total ( `results/e5_agent_tasks_claude.json` ); with the protocol in place, time and cost fell by roughly ten to twenty percent while the number of Orgalyst tool calls rose from 1.5 to 1.9, because on tool failure the assistant now tries the alternative strategies the tools offer instead of writing its own code. The distribution of tool calls is worth noting: most tasks need one to three calls and the assistant answers straight from the tool's summary, whereas group comparison and growth curve took 31 and 46 calls and 5 to 8 minutes because the assistant read the comparison and growth tables item by item after receiving the statistics and recomputed the medians in Python. This over-checking raises trust but costs time and money; in production it can be constrained through the system prompt. In the unspecified-organ task the assistant did not ask back but ran the generalist model and noted the lower accuracy in its answer, which is consistent with our convention.

## 8 Reliability analysis and limitations

**Failure cases.** The two typical segmentation failures both occur at the extreme scales of brain organoids: day-2 organoids only about 80 pixels wide are drowned by the texture of the medium in the first pass, and the second pass of refine then amplifies the error by treating a background patch as the object; day-30 organoids wider than 600 pixels are occasionally cut in two. The pdac failure is systematic: the instance density of the test images is seven times that of the training images and the instrument differs, so the model misses half of the small instances; this needs more in-domain training data or multi-scale inference for small instances, which we did not solve within the competition period. Detection counting merges two adjoining organoids into one and, conversely, counts bubbles and debris.

**Cross-organ generalisation boundary.** The leave-one-out experiment of section 7.1 gives a clear boundary: on an unseen organ AP50 is only 0.15 to 0.47. The system therefore offers unregistered organs only the generalist model with an explicit low-accuracy notice and promises no accuracy. A new organ should be fine-tuned with a few dozen annotated images (colon went from 0.405 to 0.815 with only 15 training images).

**LLM-related risks.** The assistant may choose the wrong organ, invent a pixel size, or phrase an uncertain result too confidently. Our mitigation is structural: every number comes from a tool, tool outputs carry their units and exclusion notes, the system prompt forbids estimating an unknown pixel size, and the permission gatekeeper limits file access; the E5 benchmark is the regression test for these constraints. But 12 tasks cannot cover the ways real users phrase requests, we evaluated only the Claude backend, and the gap to open-model backends is not quantified. The assistant also depends on a paid API, so reviewers reproducing the agent path need their own key; the command-line path has no such dependency and reproduces every number in this report. While preparing the demonstrations of Appendix A we also observed a counter-example: in a group comparison of day-30 brain organoids the brain model missed two of the three TH2-7 images (the very images missed in E3), and the assistant put this failure at the top of its answer, as we would want; but it then wrote its own threshold-based segmentation code to "rescue" areas for those two images and used them in the test, violating the skill's rule against writing segmentation code. Honesty and overreach appeared in the same answer, which shows that prompt-level constraints are not enough. We then added a hard block to the permission gatekeeper (CODE\_GUARDS in `agent/bioagent/permissions.py` : any Bash, REPL or file-write input containing `cv2 / skimage` thresholding, direct Cellpose or YOLO calls, regionprops and similar image-measurement code is refused with a message to use the Orgalyst tools and to report failures as failures), added the same rule to the skill, and re-ran E5 in full (section 7.5). On the same day-30 comparison task the assistant now lists the two failed images, tries only the alternative diameter strategies and the generalist model that the tools offer (neither helps), states that the p value has no interpretive value, and recommends re-imaging and annotation, without writing any code of its own. The group-comparison demonstration in the appendix uses day-16 images; the two growth-curve screenshots were recorded before the guard was added.

**Data limitations.** Only the brain dataset has a pixel calibration, so morphometry for the other organs is in pixels; the OrgLine test sets for intestine (12 images) and colon (10 images) are small and their AP50 confidence intervals are wide; all data come from published public datasets, none from organ-on-a-chip time series, and the growth-curve experiment was validated only on static brain organoid cultures. The correspondence between QC flags and real errors was evaluated only preliminarily on three organ test sets (section 7.4), with limited sample size.

## 9 Impact and future work

For bench scientists, Orgalyst shortens the path from image to report from half a day to a few minutes, and the report can go into the lab record and still be verified three months later. For method developers, the toolkit provides a framework in which weights and a diameter strategy are registered per organ; adding an organ takes one weight file and one line of configuration. Three things are most valuable next. First, organ-on-a-chip time series: organoids on a chip stay in place, so detection boxes can be linked over time for per-individual tracking instead of relying on metadata in file names as now. Second, dose response: fit four-parameter logistic curves to morphological metrics and check them against known mechanisms of action, the E6 in our design document that we did not reach. Third, integration with laboratory automation: attach the assistant as an MCP client to the scheduling interface of an imaging system so that "analyse automatically after imaging, alert on anomalies" closes the loop. The small-instance problem on pdac deserves its own effort, probably sliding-window multi-scale inference and detection-guided segmentation.

## 10 Reproduction guide

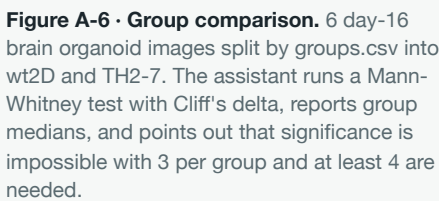
The repository README gives the full commands; here is the order with expected wall time on an RTX 5090 D. Environment: `pip install -r requirements.txt` (install torch for your CUDA version; about 10 minutes). Weights: `bash`

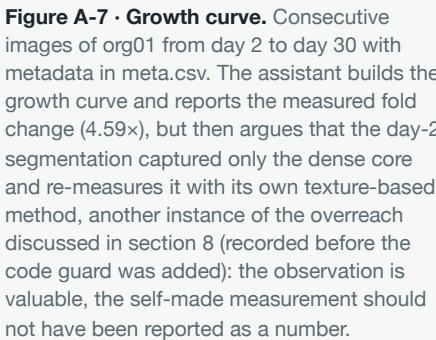
scripts/download\_weights.sh (about 200 MB; add `--mirror` in mainland China). Quick check without the dataset: `python -m orgalyst analyze --organ intestine --images any_folder_of_brightfield_images --out runs`, seconds to tens of seconds. Reproducing all tables: `download_orgline.sh` and `extract_orgline.sh` (about 25 GB, depends on bandwidth); `prepare_data.py` (15 min); `run_e1_zeroshot.sh` (20 min); `run_e1_finetune.sh` (2.5 h); `run_e1_crossorgan2.sh` (3 h); `run_e2_det.sh` (4.5 h) and `calibrate_det_conf.py` (15 min); `e3_consistency.py` (10 min); `run_cpsam_ft.sh` (needs the separate environment from `setup_cellpose4.sh`, 2.5 h); `e5_prepare.sh` and `run_agent_tasks.py` (needs agent/ and an API key, 25 min, about 6 US dollars). Every script writes to `results/`; the repository already contains all result files we produced, and every table in this report corresponds to one of them. The Dockerfile covers the command-line path.

## 11 External resources and licences

| Resource                                                                | Use                                                                                                           | Licence / source                                                            |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| OrgLine (Zenodo 16355179)                                               | All training and evaluation data                                                                              | CC-BY-4.0; not redistributed, downloaded from the original source by script |
| Original brain organoid dataset (Zenodo 10301912, Schröter et al. 2024) | Expert areas and pixel calibration for E3                                                                     | CC-BY-4.0                                                                   |
| Cellpose 3 (cyto3 weights, size_cyto3)                                  | Segmentation backbone and initial weights                                                                     | BSD-3, Stringer & Pachitariu                                                |
| Cellpose-SAM (cpsam / cpsam_v2)                                         | Comparison experiment                                                                                         | BSD-3                                                                       |
| Ultralytics YOLO11m                                                     | Counting by detection                                                                                         | AGPL-3.0 (fine-tuned weights inherit it)                                    |
| scikit-image, scipy, pandas, matplotlib                                 | Measurement, statistics, plotting                                                                             | BSD / MIT                                                                   |
| Claude Agent SDK, Claude (claude-opus-5)                                | Assistant layer and the E5 benchmark                                                                          | Anthropic commercial API; the command-line path does not depend on it       |
| MCP (FastMCP)                                                           | Tool interface                                                                                                | MIT                                                                         |
| Claude Code                                                             | Coding assistant during development (all code, documents and experiment scripts were reviewed by the authors) | Anthropic                                                                   |

Project code is released under MIT; fine-tuned weights are hosted on Hugging Face (XiaoyanLi/orgalyst-weights) and inherit the licences of their base models.





**Figure A-10 • Empty folder.** The folder contains no images. The assistant says so plainly instead of returning a plausible-looking number.

